Bài 2.15 – PCA (Phân tích thành phần chính)

> **Giai đoạn 2 – Machine Learning cổ điển** · Thời lượng: 2 buổi · Hình thức mở đầu: *So sánh ví von*

🎯 Sau bài này bạn sẽ

- Hiểu PCA: tìm các **hướng nhìn** giữ được nhiều thông tin nhất, rồi bỏ bớt hướng ít thông tin.
- Đọc được `explained_variance_ratio_`, chọn số thành phần cần giữ.
- Dùng PCA để **vẽ dữ liệu nhiều chiều** lên mặt phẳng 2D.
- Dùng PCA để **nén** dữ liệu và tăng tốc mô hình, và biết khi nào **không nên** dùng.
- Biết vì sao PCA **bắt buộc chuẩn hóa**.

Chia buổi:

- **Buổi 1:** Mục 1 → 4 (trực giác, ví dụ 2 chiều, phương sai giải thích).
- **Buổi 2:** Mục 5 → 9 (30 chiều → 2 chiều, chọn số thành phần, nén ảnh, bài tập).

---

1. So sánh ví von: Chụp ảnh một chiếc ấm trà

Bạn muốn chụp **một tấm ảnh** (2 chiều) để người xem nhận ra một **vật thể 3 chiều**: chiếc ấm trà.



Góc chụp **ngang thân** giữ lại nhiều thông tin nhất: thấy được vòi, quai, thân. Góc từ trên xuống làm mọi thứ "dẹt" lại, mất gần hết đặc điểm.

PCA làm đúng việc đó cho dữ liệu: tìm **góc nhìn** (hướng chiếu) sao cho khi "chụp" dữ liệu nhiều chiều xuống ít chiều hơn, **các điểm vẫn trải rộng, khác biệt nhau nhiều nhất**. Trải rộng = còn giữ thông tin để phân biệt.

---

2. Khái niệm

- **Dimensionality reduction (Giảm chiều):** Biến dữ liệu nhiều cột thành ít cột hơn, giữ lại phần lớn thông tin.
- **PCA – Principal Component Analysis (Phân tích thành phần chính):** Phương pháp giảm chiều phổ biến nhất. Tìm các **trục mới** (gọi là **thành phần chính**, principal components – PC):
  - **PC1:** hướng mà dữ liệu **trải rộng nhất** (phương sai lớn nhất, bài 1.7).
  - **PC2:** hướng trải rộng nhất **vuông góc** với PC1.
  - PC3, PC4...: tiếp tục như vậy, mỗi cái vuông góc với tất cả cái trước.
- Mỗi thành phần chính là một **tổ hợp tuyến tính** của các cột gốc (một tích vô hướng, bài 1.8).

Vì sao cần giảm chiều?

Lý do	Ví dụ
Vẽ được dữ liệu nhiều chiều	30 chỉ số tế bào → 2 trục để vẽ scatter
Chống **lời nguyền số chiều** (bài 2.7)	KNN, K-Means hoạt động tốt hơn với ít chiều
Gộp các cột **trùng thông tin** (đa cộng tuyến, bài 2.6)	5 cột "tổng" của dữ liệu nhà tương quan 0,9
Nén, tăng tốc	Ảnh 784 điểm ảnh → 50 số

---

3. Ví dụ 2 chiều: điểm Toán và Lý

3.1. Dữ liệu

6 học sinh, điểm Toán và Lý tương quan rất mạnh:

```
import numpy as np
import pandas as pd
from sklearn.decomposition import PCA

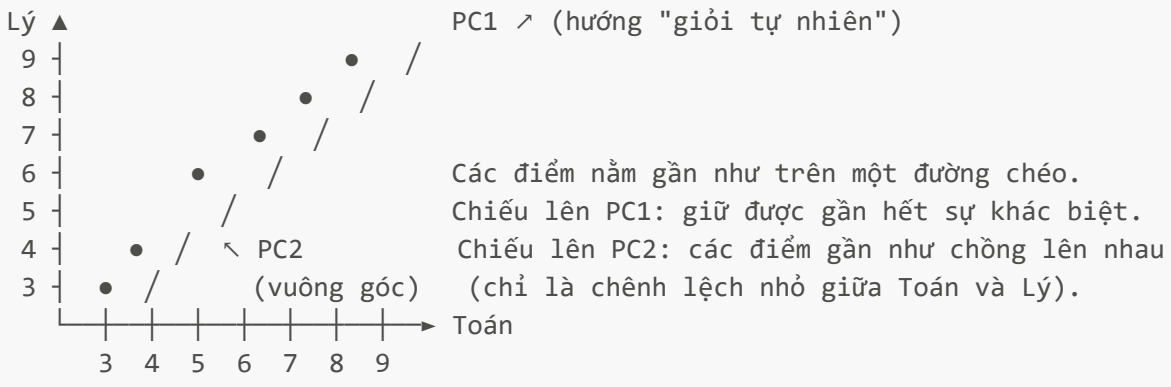
hs = pd.DataFrame({"toan": [9, 8, 6, 5, 3, 7], "ly": [8.5, 8, 6.5, 4.5, 3.5, 7]})
print(hs.corr().round(3))
```

Kết quả mẫu:

```
       toan     ly
toan  1.000  0.981
ly    0.981  1.000
```

Tương quan 0,981 (bài 1.7): hai cột gần như "nói cùng một điều": học sinh giỏi khối tự nhiên hay không.

3.2. Trực giác



3.3. Code

```
p = PCA(n_components=2).fit(hs)          # PCA tự trừ trung bình mỗi cột trước khi
tính
print(p.components_.round(3))            # mỗi hàng là một thành phần chính
(hướng)
print(p.explained_variance_ratio_.round(4))   # tỉ lệ phương sai mỗi thành phần
giữ lại
print(p.transform(hs).round(2))          # tọa độ của 6 học sinh trên hệ trục mới
```

Kết quả mẫu:

```
[[ 0.74   0.672]
 [-0.672  0.74 ]]
[0.9905 0.0095]
[[ 3.43 -0.19]
 [ 2.35  0.11]
 [-0.13  0.35]
 [-2.22 -0.46]
 [-4.37  0.14]
 [ 0.94  0.05]]
```

Đọc:

- **PC1 = 0,74 × (Toán − TB) + 0,672 × (Lý − TB)**: gần như "tổng điểm Toán + Lý". Đây là trục "giỏi tự nhiên".
- **PC2 = −0,672 × Toán + 0,74 × Lý** (đã trừ trung bình): "Lý hơn Toán bao nhiêu".
- PC1 giữ **99,05%** phương sai. PC2 chỉ **0,95%**.
- Học sinh 1 (9 Toán, 8,5 Lý) có PC1 = 3,43: giỏi nhất. Học sinh 5 (3 Toán, 3,5 Lý) có PC1 = −4,37: yếu nhất.

Tính thử PC1 của học sinh 1: trung bình Toán = 38/6 ≈ 6,333; trung bình Lý = 38/6 ≈ 6,333. 0,74 × (9 − 6,333) + 0,672 × (8,5 − 6,333) = 0,74 × 2,667 + 0,672 × 2,167 ≈ 1,973 + 1,456 ≈ **3,43**. ✓

3.4. Giảm từ 2 xuống 1 chiều, rồi khôi phục

```
p1 = PCA(n_components=1).fit(hs)
khoi_phuc = p1.inverse_transform(p1.transform(hs))     # nén xuống 1 số, rồi "giải
nén" ngược lại
print(np.column_stack([hs.values, khoi_phuc.round(2)]))  # [Toán, Lý gốc | Toán,
Lý khôi phục]
```

Kết quả mẫu:

```
[[9.   8.5  8.87 8.64]
 [8.   8.   8.08 7.92]
 [6.   6.5  6.23 6.24]
 [5.   4.5  4.69 4.84]
```

```
[3.   3.5  3.1  3.39]
[7.   7.   7.03 6.97]]
```

Chỉ lưu **1 con số** cho mỗi học sinh (thay vì 2), vẫn khôi phục được điểm gần đúng (sai lệch tối đa khoảng 0,3 điểm). Phần mất đi chính là 0,95% phương sai của PC2.

---

4. Phương sai giải thích (Explained variance)

- **Explained variance ratio:** Tỉ lệ "thông tin" (phương sai) mà mỗi thành phần chính giữ lại. Tổng của tất cả các thành phần = 100%.
- Thành phần chính luôn được **sắp giảm dần**: PC1 ≥ PC2 ≥ PC3...
- **Cách chọn số thành phần:** giữ đủ thành phần để **tổng tích lũy** đạt 90–95% (hoặc 2–3 thành phần nếu chỉ để vẽ).

---

5. Dữ liệu thật: 30 chỉ số tế bào ung thư → 2 chiều

5.1. Bắt buộc chuẩn hóa

PCA tìm hướng có **phương sai lớn nhất**. Nếu một cột có đơn vị lớn (diện tích tế bào: hàng trăm, hàng nghìn), nó sẽ "chiếm" PC1 chỉ vì con số lớn.

```
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler

bc = load_breast_cancer(as_frame=True)

p_raw = PCA().fit(bc.data)                          # ❌ không chuẩn hóa
print(p_raw.explained_variance_ratio_[:2].round(4))
print(pd.Series(p_raw.components_[0], index=bc.data.columns).sort_values(key=abs,
ascending=False).head(2).round(3))
```

Kết quả mẫu:

```
[0.982  0.0162]
worst area    0.852
mean area     0.517
dtype: float64
```

Không chuẩn hóa: PC1 "giải thích" 98,2%, nhưng thực chất PC1 **chỉ là cột diện tích** (worst area, mean area), vì diện tích có con số lớn nhất. 28 cột khác bị bỏ qua. Kết quả vô nghĩa.

5.2. Có chuẩn hóa

```
Xs = StandardScaler().fit_transform(bc.data)      # ✅ chuẩn hóa trước
pca = PCA().fit(Xs)
print(pca.explained_variance_ratio_[:5].round(3), "| 2 PC đầu:",
pca.explained_variance_ratio_[:2].sum().round(3))

tich_luy = np.cumsum(pca.explained_variance_ratio_)                    # tổng
tích lũy
print("Số PC cần để đạt 80%, 90%, 95%, 99%:",
      [int(np.argmax(tich_luy >= t)) + 1 for t in [0.8, 0.9, 0.95, 0.99]])
```

Kết quả mẫu:

```
[0.443 0.19  0.094 0.066 0.055] | 2 PC đầu: 0.632
Số PC cần để đạt 80%, 90%, 95%, 99%: [5, 7, 10, 17]
```

- PC1 giữ 44,3%, PC2 giữ 19%. Hai chiều giữ được **63,2%** thông tin của 30 chiều.
- Chỉ cần **10 thành phần** (thay vì 30 cột) để giữ 95% thông tin. 30 cột này có rất nhiều thông tin trùng lặp (bán kính, chu vi, diện tích của cùng một tế bào thì tương quan với nhau).

5.3. PC1 nghĩa là gì?

```
pc1 = pd.Series(PCA(n_components=2).fit(Xs).components_[0], index=bc.data.columns)
print(pc1.sort_values(key=abs, ascending=False).head(5).round(3))
```

Kết quả mẫu:

```
mean concave points     0.261
mean concavity          0.258
worst concave points    0.251
mean compactness        0.239
worst perimeter         0.237
dtype: float64
```

PC1 là tổ hợp gần như đều của các chỉ số về **độ lõm, độ đặc, kích thước** tế bào. Có thể đặt tên: "mức độ bất thường của tế bào". (Không phải lúc nào cũng đặt tên được cho thành phần chính. Đây là điểm yếu của PCA: các trục mới thường **khó giải thích**.)

5.4. Vẽ 30 chiều lên mặt phẳng

```
import matplotlib.pyplot as plt

z2 = PCA(n_components=2).fit_transform(Xs)          # mỗi mẫu còn 2 số
plt.figure(figsize=(6, 5))
```

```
plt.scatter(z2[:, 0], z2[:, 1], c=bc.target, s=8, cmap="coolwarm")   # tô màu theo
nhãn thật
plt.xlabel("PC1 (44,3%)"); plt.ylabel("PC2 (19%)")
plt.title("569 mẫu sinh thiết, từ 30 chiều xuống 2 chiều")
plt.show()
```

Bạn sẽ thấy: hai đám mây điểm khá tách biệt theo trục PC1: mẫu lành tính tụ lại một bên, mẫu ác tính trải rộng về bên kia. PCA **không hề biết nhãn**, nhưng chỉ bằng cách tìm hướng trải rộng nhất, nó đã làm lộ ra cấu trúc phân biệt hai loại.

> 💡 PCA là **học không giám sát**: không dùng y. Ở đây ta chỉ dùng y để **tô màu** khi vẽ, để kiểm tra.

---

6. PCA trước một mô hình

```
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression

Xtr, Xte, ytr, yte = train_test_split(bc.data, bc.target, test_size=0.25,
random_state=42, stratify=bc.target)
for k in [1, 2, 5, 10, 30]:
    m = make_pipeline(StandardScaler(), PCA(n_components=k),
LogisticRegression(max_iter=1000)).fit(Xtr, ytr)
    print(f"{k:>2} thành phần: acc test = {m.score(Xte, yte):.3f}")
```

Kết quả mẫu:

```
 1 thành phần: acc test = 0.909
 2 thành phần: acc test = 0.944
 5 thành phần: acc test = 0.965
10 thành phần: acc test = 0.979
30 thành phần: acc test = 0.986
```

- Chỉ **1 con số** (PC1) đã phân loại đúng 91%.
- 10 thành phần đạt 97,9%, gần bằng dùng đủ 30 (98,6%).
- Ở đây giảm chiều **không làm tốt hơn**, vì Logistic Regression với 30 cột vốn đã ổn. PCA giúp khi: mô hình chậm, nhiều cột trùng thông tin, hoặc thuật toán dựa trên khoảng cách (KNN, K-Means).

> ⚠️ Đặt PCA **trong pipeline**, sau StandardScaler. PCA cũng "học" (học hướng các trục) nên phải fit chỉ trên train (bài 1.11).

---

7. Nén ảnh chữ số viết tay

Bộ **Digits** (có sẵn): 1.797 ảnh chữ số viết tay, mỗi ảnh 8 × 8 = **64 điểm ảnh** (64 chiều).

```
from sklearn.datasets import load_digits

dg = load_digits()
print(dg.data.shape)
pd_ = PCA().fit(dg.data)          # ảnh: các điểm ảnh cùng thang đo 0–16, không cần chuẩn hóa
tl = np.cumsum(pd_.explained_variance_ratio_)
print("Số PC cần để đạt 50%, 80%, 90%, 95%:", [int(np.argmax(tl >= t)) + 1 for t in [0.5, 0.8, 0.9, 0.95]])
print(PCA(n_components=0.95).fit(dg.data).n_components_)   # n_components < 1: tự chọn số PC đạt tỉ lệ đó

fig, axes = plt.subplots(1, 5, figsize=(10, 2.5))
for ax, k in zip(axes, [2, 5, 10, 20, 64]):
    p = PCA(n_components=k).fit(dg.data)
    anh = p.inverse_transform(p.transform(dg.data[:1]))       # nén ảnh đầu tiên xuống k số rồi khôi phục
    ax.imshow(anh.reshape(8, 8), cmap="gray_r"); ax.set_title(f"{k} PC");
ax.axis("off")
plt.show()
```

Kết quả mẫu:

```
(1797, 64)
Số PC cần để đạt 50%, 80%, 90%, 95%: [5, 13, 21, 29]
29
```

Bạn sẽ thấy: ảnh chữ số "0" khôi phục từ **2 PC** chỉ là một vệt mờ; từ **10 PC** đã nhận ra hình số 0; từ **20 PC** gần như giống bản gốc 64 PC.

Nén 64 số xuống **29 số** (giảm hơn một nửa) mà vẫn giữ 95% thông tin. Ý tưởng "nén thông tin vào ít chiều" sẽ xuất hiện lại ở **autoencoder** và **embedding** trong Deep Learning (bài NLP2.1).

---

8. Ưu, nhược điểm và lưu ý

Ưu điểm	Nhược điểm
Nhanh, đơn giản, có sẵn mọi nơi	Thành phần mới **khó giải thích** (PC1 = 0,26 × độ lõm + 0,24 × chu vi + ...)
Gộp các cột trùng thông tin	Chỉ bắt được quan hệ **tuyến tính**
Giúp vẽ, tăng tốc, chống lời nguyền số chiều	Bắt buộc chuẩn hóa (khi cột khác đơn vị)
	Hướng "trải rộng nhất" **chưa chắc** là hướng tốt nhất để phân loại

Khi nào KHÔNG nên dùng PCA:

- Khi cần giải thích mô hình cho người dùng (ngân hàng, y tế).
- Với mô hình cây (Random Forest, XGBoost): chúng tự chọn đặc trưng tốt, PCA thường không giúp, đôi khi còn làm hại.

Muốn vẽ dữ liệu phức tạp hơn? Thử **t-SNE** hoặc **UMAP**: giữ được cấu trúc "ai gần ai" phi tuyến, tạo hình đẹp hơn PCA. Nhưng **chỉ dùng để vẽ**, không dùng làm đặc trưng cho mô hình.

---

9. Bài tập

1. Trong ví dụ Toán–Lý (mục 3.3), **tính tay** PC1 của học sinh 5 (3 Toán, 3,5 Lý). So sánh với −4,37.
2. Áp dụng PCA (có chuẩn hóa) cho 5 cột "tổng" của dữ liệu nhà: `tong_phong`, `tong_phong_ngu`, `dan_so`, `so_ho`, `thu_nhap`. PC1 giữ bao nhiêu %? Hệ số của PC1 cho thấy điều gì?
3. Dùng bộ Digits: `make_pipeline(PCA(n_components=k), KNeighborsClassifier())` với k = 5, 10, 20, 64. Accuracy và **thời gian dự đoán** thay đổi thế nào?
4. Vẽ scatter PC1–PC2 của bộ Digits, tô màu theo nhãn (0–9). Những chữ số nào bị chồng lên nhau nhiều? Có hợp lý không?

Đáp án

> Mẹo Notion: chọn tiêu đề "Đáp án" → `Turn into → Toggle heading 3`.

1. 0,74 × (3 − 6,333) + 0,672 × (3,5 − 6,333) = 0,74 × (−3,333) + 0,672 × (−2,833) ≈ −2,467 − 1,904 ≈ **−4,37**. ✓
2. Code:

```
df = nap_du_lieu_nha()
cot5 = ["tong_phong", "tong_phong_ngu", "dan_so", "so_ho", "thu_nhap"]
p5 = PCA().fit(StandardScaler().fit_transform(df[cot5]))
print(p5.explained_variance_ratio_.round(3))
print(pd.Series(p5.components_[0], index=cot5).round(3))
```

Kết quả thật: PC1 giữ **75,1%**, PC2 giữ 20,4%. PC1 có hệ số ≈ 0,49–0,51 cho cả 4 cột tổng và chỉ 0,057 cho thu nhập; PC2 có hệ số 0,982 cho thu nhập. Vậy PC1 gần như là trung bình của 4 cột tổng: PC1 chính là "**quy mô khu dân cư**". Thu nhập gần như nằm riêng ở một thành phần khác. PCA tự phát hiện ra 2 "khái niệm" ẩn: quy mô và mức sống. 3. Code:

```
import time
from sklearn.neighbors import KNeighborsClassifier
Xtr_d, Xte_d, ytr_d, yte_d = train_test_split(dg.data, dg.target, test_size=0.25,
random_state=42, stratify=dg.target)
for k in [5, 10, 20, 64]:
    m = make_pipeline(PCA(n_components=k), KNeighborsClassifier()).fit(Xtr_d,
ytr_d)
    t0 = time.time(); acc = m.score(Xte_d, yte_d)
    print(k, round(acc, 3), f"{time.time() - t0:.3f} giây")
```

Accuracy tăng nhanh rồi bão hòa khoảng 20 PC; dự đoán nhanh hơn với ít chiều. Ghi lại kết quả thật. 4. Các chữ số có hình dạng giống nhau (như 3, 5, 8 hoặc 1, 7) thường chồng lên nhau; chữ số 0 và 6 thường tách khá rõ. Hợp lý, vì PCA chỉ giữ 2 hướng trải rộng nhất (28,5% thông tin), không đủ để phân biệt mọi chữ số.

---

10. Flashcard

Mặt trước	**Mặt sau**
PCA làm gì?	Tìm các hướng mới (thành phần chính) giữ phương sai lớn nhất, để giảm chiều.
PC1 là gì?	Hướng dữ liệu trải rộng nhất.
PC2 liên quan PC1 thế nào?	Vuông góc với PC1, trải rộng nhất trong phần còn lại.
`explained_variance_ratio_`?	Tỉ lệ phương sai mỗi thành phần giữ lại.
Chọn bao nhiêu thành phần?	Đủ để tổng tích lũy đạt 90–95% (hoặc 2–3 để vẽ).
PCA có cần chuẩn hóa?	Có, khi các cột khác đơn vị.
PCA có dùng nhãn y?	Không; là học không giám sát.
`inverse_transform` làm gì?	Khôi phục dữ liệu gần đúng từ các thành phần đã giữ.
Nhược điểm lớn của PCA?	Thành phần mới khó giải thích; chỉ bắt quan hệ tuyến tính.
`PCA(n_components=0.95)` nghĩa là?	Tự chọn số thành phần ít nhất để giữ 95% phương sai.

Bài tiếp theo: 2.16 – Feature Engineering.